

Paralelização Automática de Laços Irregulares

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais

Relatório Final da Disciplina Projeto Orientado em Computação II
Caráter Científico
1º semestre de 2016

Aluno

Pedro Ramos
pedroramos@dcc.ufmg.br

Orientador

Fernando M. Q. Pereira
fernando@dcc.ufmg.br

Julho de 2016

Resumo

Paralelização automática de código é um assunto popular e com crescente foco da indústria, que opta por investir muito em paralelismo para atingir bom alto desempenho. Porém, a maior parte dos trabalhos em paralelização automática de código lidam com laços regulares e paralelismo de dados. Apresentamos o Task Miner, um algoritmo de análise estática capaz de extrair paralelismo de laços irregulares. Para tal, geramos o grafo de dependências em nível de instrução do programa e o analisamos. A ideia chave do algoritmo reside em extrair as chamadas Hélices do laço, pedaços de código que estão dentro de um laço irregular mas que não geram nenhuma dependência entre iterações, e portanto podem ser paralelizados por tarefas.

Este é um trabalho ainda em desenvolvimento e até a entrega deste relatório ainda não havia sido completado. Com o Task Miner, esperamos poder analisar aplicações já existentes e encontrar oportunidades escondidas de paralelismo em laços irregulares que possam conferir bons ganhos de desempenho caso paralelizadas. Além disso, pretendemos testá-lo com benchmarks importantes na área de paralelização de código e procuraremos definir uma métrica para determinar quais hélices se tornarão tarefas.

1 Introdução

Paralelização automática de código é atualmente um tópico de pesquisa em Ciência da Computação muito popular, em grande parte devido ao advento dos processadores no mercado com múltiplas unidades de processamento, que permitem uma computação altamente distribuída, e da mudança de paradigma da indústria, que hoje opta por investir muito em paralelismo para atingir bom desempenho [1]. De fato, paralelismo pode proporcionar ganhos consideráveis

em desempenho, porém o fardo de encontrar as oportunidades para se paralelizar o programa ainda cabe ao programador. O programador deve analisar o código, adaptar as estruturas e métodos para adequar-se a um paradigma paralelo. Essa tarefa não é trivial e está sujeita a erros [2]. Os modelos de programação paralela atuais utilizam *threads* que executam sobre um ambiente de memória compartilhada, porém, é notoriamente difícil escrever programas de múltiplas *threads* [3]. Neste trabalho propomos o Task Miner, um algoritmo que é capaz de encontrar paralelismo em laços irregulares automaticamente, e anotá-los com diretivas de paralelismo de tarefas. A ideia chave do Task Miner é que uma tarefa é uma "Hélice" em um laço, um bloco de código que não depende das iterações do laço, e que portanto é uma candidata a se tornar uma tarefa. O algoritmo consiste então, basicamente, em encontrar esses pedaços de código em laços irregulares.

1.1 Paralelismo de Dados e de Tarefas

Há tipicamente duas formas de paralelismo em alto nível: paralelismo de dados e paralelismo de tarefas [4], [5]. No paralelismo de dados, uma mesma computação é aplicada sobre diferentes porções do mesmo dado. Isso ocorre muito com operações *stencil*, que geralmente envolvem processamento gráfico, no qual uma computação simples deve ser aplicada a uma larga quantidade de dados e não há dependência entre esses dados, de forma que a operação pode ser feita concomitantemente. Na Figura 1 podemos ver um exemplo de paralelismo de dados. A variável *sum* tem, a cada iteração do laço, seu valor incrementado com o valor de uma posição diferente do arranjo *v*. Esta operação é uma redução, pois o operador de soma é associativo, portanto não importa a ordem em que façamos a operação de soma dos elementos, o resultado será o mesmo. Paralelizar este laço implicaria em aplicar a mesma operação (soma) em diferentes porções do mesmo arranjo. O paralelismo de dados baseia-se na natureza distribuída do dado.

Já na Figura 2, temos um exemplo clássico de paralelismo de tarefas. Cada chamada recursiva dentro do método *mergesort* é independente, ou seja, ambas podem executar em paralelo. O paralelismo por tarefas é baseado na natureza distribuída do código independente.

```
for (unsigned i=0; i < v.size(); i++)  
{  
    sum += v[i];  
}
```

Figura 1: Exemplo de paralelismo de dados: redução de soma no vetor.

1.2 Laços Regulares e Irregulares

De certa forma, dizemos que paralelismo de dados é mais comum em aplicações regulares, nas quais podemos prever com certa exatidão quais operações serão

```

void merge_sort(int* vec, int size, int *tmp)
{
    if(size == 1)
    {
        return;
    }
    merge_sort(vec, size/2, tmp);
    merge_sort(vec + (size/2), size-(size/2), tmp);
    merge(vec, size, tmp);
}

```

Figura 2: Exemplo de paralelismo de tarefas: mergesort.

executadas e quantas vezes um laço vai iterar. Trabalhos já foram feitos para paralelizar automaticamente aplicações regulares, como por exemplo laços canônicos [1]. Precisamente neste trabalho, dedicamos nossa atenção a encontrar paralelismo em laços irregulares, ou seja, encontrar paralelismo por tarefas [6]. Desenvolvemos um algoritmo para encontrar código independente para ser paralelizado por tarefas em laços irregulares. Achar paralelismo de tarefas também não é algo trivial. A parte mais complexa em achar as tarefas reside em resolver as dependências entre as tarefas [7]. Voltaremos a essa questão na seção 3.

Já que a maior parte do tempo de um programa é geralmente gasto em laços [8], é normal que haja um foco em otimizações de laços. Na literatura, laços são frequentemente classificados como DOALL ou DOACROSS [9]. Um laço é considerado DOALL se não possui dependências entre iterações, ou seja, pode ser paralelizado facilmente. Basicamente, se equivale ao conceito de regularidade: um laço DOALL é um laço regular. Analogamente, um laço DOACROSS é um laço irregular, ou seja, possui dependências entre as iterações do laço. Paralelizá-lo é mais desafiador [10]. Pesquisadores vem investigando o paralelismo de laços irregulares (DOACROSS) por um longo tempo [9], [11], [12].

1.3 Anotando Programas com OpenMP

Para evitar transformar o programa original de forma a se adequar a um paradigma paralelo, podemos utilizar sistemas de anotações como OpenMP [13]. Essas anotações atuam como uma meta-linguagem e provêm ao programador a capacidade de conferir semântica paralela à sintaxe que originalmente foi escrita para executar sequencialmente. Dessa forma, aplicações já existentes podem aproveitar os benefícios do hardware paralelo atual sem precisar alterar sua sintaxe, basta que tenham suas partes que podem ser paralelizadas anotadas com uma diretiva. Na Figura 3 as duas chamadas independentes do *mergesort* foram anotadas com a diretiva de paralelismo de tarefas, tornando assim um programa sintaticamente sequencial em paralelo sem alterar a sintaxe do mesmo.

Embora sistemas de anotações como OpenMP facilitem a implementação do paradigma paralelo, cabe ainda ao programador o fardo de encontrar as oportunidades de paralelismo e inserir as diretivas. A grande motivação por trás do nosso trabalho é

A grande motivação por trás deste trabalho reside em retirar do programador o fardo de encontrar o paralelismo. Apresentamos o Task Miner, uma

```

void merge_sort(int* vec, int size, int *tmp)
{
    if(size == 1)
    {
        return;
    }
    #pragma omp task firstprivate(vec, size, tmp)
    merge_sort(vec, size/2, tmp);
    #pragma omp task firstprivate(vec, size, tmp)
    merge_sort(vec + (size/2), size-(size/2), tmp);

    #pragma omp taskwait
    merge(vec, size, tmp);
}

```

Figura 3: Exemplo de diretivas OpenMP para paralelizar o mergesort por tarefas.

análise estática que procura por oportunidades de paralelismo em código C e C++. Mais precisamente, o Task Miner procura por oportunidades em laços irregulares que possam ser anotadas com a diretiva OpenMP de paralelismo de tarefas. Com o Task Miner, apresentamos uma forma de extrair-se as oportunidades de paralelismo em laços irregulares que possam ser anotadas com a diretiva OpenMP de paralelismo de tarefas, dessa forma proporcionando assim um ganho em desempenho. O Task Miner explora o programa e expõe oportunidades desperdiçadas em que, se paralelizadas, poderiam conferir ganhos em desempenho.

2 Visão Geral

Chamamos o ato de analisar os programas e procurar por oportunidades de paralelismo por tarefas de **Mineração de Tarefas**. O ato de procurar por tarefas em laços irregulares tem vários desafios. O maior desafio é que, em ambiente de execução, criar e manter uma tarefa é caro. O maior custo relacionado ao paralelismo por tarefas é o gerenciamento da árvore de dependências das tarefas [3].

Anotamos pequenos programas com diretivas OpenMP e observamos alguns padrões que conferem bons resultados (ganhos em desempenho) quando paralelizados.

2.1 Mineração de Tarefas: Padrões Observados

Observamos alguns padrões que conferem bons em desempenho se paralelizados por tarefas. Os mesmos estão ilustrados nas Figuras 4, 5 e 6. Na Figura 4 observamos o padrão recursivo. O algoritmo de ordenação *mergesort* cai nesta categoria. Porém, a mera constatação deste padrão não é suficiente. Como o custo de criação e manutenção de uma tarefa é alto em tempo de execução, é preciso que a tarefa realize uma computação cara o suficiente para compensar o tempo gasto na manutenção das tarefas.

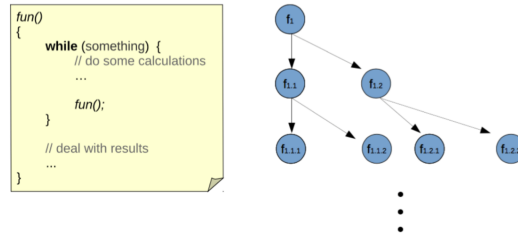


Figura 4: Padrão recursivo

Nas Figuras 5 e 6 observamos os padrões em laços, que são o foco do algoritmo. Ambos os laços nas duas figuras são irregulares. Na Figura 5, o bloco de código A é independente, ou seja, não possui dependências entre as iterações. Portanto posso executar várias iterações do bloco A em várias tarefas paralelamente. Porém o bloco B possui dependência entre iterações do laço, e portanto não pode ser paralelizado. Na Figura 6 o contrário ocorre: o bloco A é dependente e o bloco B é independente. B, nesse caso, é uma forte candidata a se tornar uma tarefa.



Figura 5: Bloco (A) é independente, bloco (B) possui uma dependência.

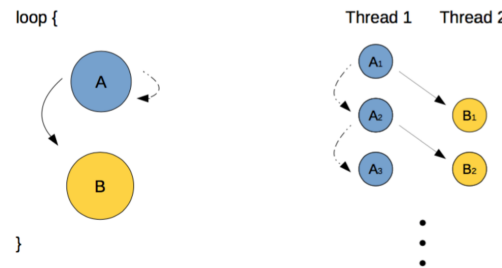


Figura 6: Bloco (B) é independente, bloco (A) possui uma dependência.

Para observarmos estes padrões em um código, é preciso gerar o **grafo de**

```

void task(int* v, int i) {
    int tmp = v[i] * v[i];
    tmp ^= i;

    v[i] = tmp & 0x0000FFFF;
}

int main(int argc, char** argv) {
    int N = atoi(argv[1]);
    int *v = (int*) malloc(N * sizeof(int));
    int *x = (int*) malloc(N * sizeof(int));
    int i, sum=0;

    // this is a do across loop:
    for (i = 0; i < N; i++) {
        sum += x[i];
        task(v, i);
    }

    return sum;
}

```

Figura 7: Exemplo motivacional. O laço na função *main* é irregular pois realiza uma redução na variável *sum*, porém a chamada à função *task* é independente e pode ser paralelizada.

dependências do programa. O que nos leva ao desenvolvimento do algoritmo e sua ideia chave, a serem descritos na próxima seção.

3 Desenvolvimento do Trabalho

Como dito na seção anterior, para encontrar os padrões estudados, precisamos gerar o grafo de dependências do programa. Mais precisamente, precisamos gerar o grafo de dependências dos laços contidos no programa.

3.1 Grafos de Dependência

O grafo de dependência de um programa (PDG - *Program Dependence Graph*) é uma representação intermediária do programa que torna explícitas todas as dependências de dados e de controle para cada instrução em um programa. Muitas otimizações tradicionais operam mais eficientemente sobre o PDG [14].

Exemplo motivacional. Considere o exemplo mostrado na Figura 7. Neste simples código, há um laço na função *main*. Este laço é irregular, pois contém uma dependência entre iterações na variável *sum*, sobre a qual é realizada uma operação de redução. Logo abaixo a redução, temos uma chamada para a função *task*. Observamos que a função *task* é uma boa candidata a se tornar uma tarefa, pois ela não gera dependências de saída para a operação

de um laço no grafo de dependências. É como se o ciclo fosse um moinho e tudo que saísse do centro, fossem suas hélices [15]. De uma forma geral, estas hélices que saem do ciclo da variável de indução são fortes candidatas a se tornarem tarefas.

A Figura 9 descreve a ideia chave por trás do TaskMiner: encontrar as hélices do laço irregular. Essas hélices são fortes candidatas a se tornarem tarefas. Esta é a ideia chave do algoritmo e um estágio importante no mesmo, porém não é suficiente. Para determinar se a hélice deve ou não se tornar uma tarefa, precisaremos de uma métrica para medir o potencial da tarefa, e precisaremos também extrair as variáveis de leitura e escrita na hélice para determinar os conjuntos de entrada e de saída da tarefa.

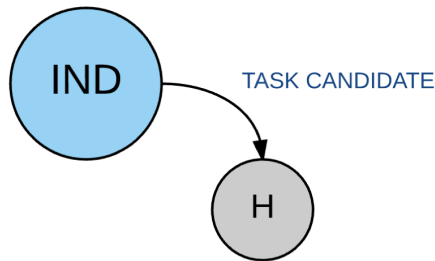


Figura 9: A Hélice de um laço é uma candidata a tarefa.

3.2 O Algoritmo

O algoritmo do Task Miner segue o pseudocódigo abaixo. O Task Miner recebe como entrada um código escrito em C/C++ e realiza a seguinte sequência de ações:

1. Gere o Grafo de Dependências do Programa (DPG).
 - (a) Encontre os Componentes Fortemente Conexos do DPG.
2. Encontre as variáveis de indução dos laços. Este passo é através de evolução escalar [8], [16].
3. Para cada variável de indução dos laços, encontre as Hélices de cada. As Hélices são candidatas a se tornarem tarefas.
4. Para cada Hélice encontrada: caso ela seja uma boa candidata a se tornar uma Tarefa, faça:
 - (a) Resolva os conjuntos INS e OUTS (Variáveis lidas e variáveis escritas dentro daquela região da Hélice).

5. Anote o programa original com diretivas OpenMP na região correspondente à Hélice escolhida.

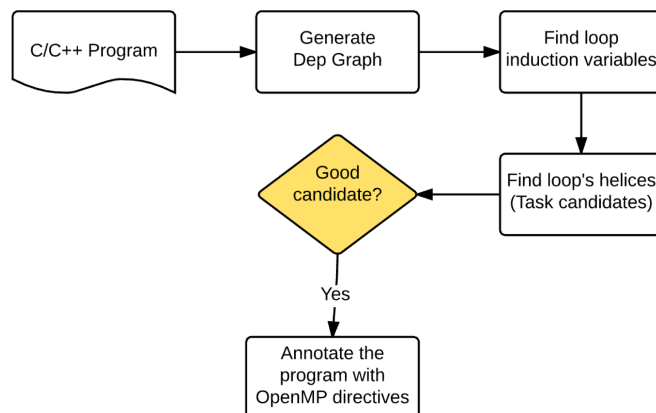


Figura 10: Task Miner: o Algoritmo

A Figura 10 representa com mais clareza de ideias o fluxograma do algoritmo.

4 Conclusões e Trabalhos Futuros

Este é um trabalho em desenvolvimento com previsão de término para Agosto e até a entrega deste relatório o Task Miner ainda não havia sido concluído. De forma geral, pretendemos com o Task Miner propor uma maneira de se extrair paralelismo em laços irregulares encontrando as Hélices do laço. Além disso, pretendemos testar nosso algoritmo em Benchmarks já conhecidos e analisar quantas oportunidades de paralelismo podemos encontrar nesses termos em relação ao que já foi feito.

Além disso, parte crucial do Task Miner é determinar uma métrica para as tarefas. Todas as Hélices não podem se tornar tarefas por padrão pois algumas tarefas seriam pequenas demais, causando um prejuízo devido ao custo de manutenção de tarefas em tempo de execução. A definição da métrica pode estar relacionada ao número de instruções da Hélice ou ao número de execuções do laço interno, quando a Hélice possuir um outro laço interno.

Referências

- [1] L. L. P. Da Mata, F. M. Q. Pereira, and R. Ferreira, “Automatic parallelization of canonical loops,” *Science of Computer Programming*, vol. 78, no. 8, pp. 1193–1206, 2013.

- [2] T. Bai, C. Ding, and P. Li, “Assessing safe task parallelism in spec 2006 int,” in *Cluster, Cloud and Grid Computing (CCGrid), 2015 15th IEEE/ACM International Symposium on*. IEEE, 2015, pp. 402–411.
- [3] P. Pratikakis, H. Vandierendonck, S. Lyberis, and D. S. Nikolopoulos, “A programming model for deterministic task parallelism,” in *Proceedings of the 2011 ACM SIGPLAN Workshop on Memory Systems Performance and Correctness*. ACM, 2011, pp. 7–12.
- [4] D. B. Skillicorn, “Models for practical parallel computation,” *International Journal of Parallel Programming*, vol. 20, no. 2, pp. 133–158, 1991.
- [5] D. E. Culler, J. P. Singh, and A. Gupta, *Parallel computer architecture: a hardware/software approach*. Gulf Professional Publishing, 1999.
- [6] M. Kulkarni, M. Burtcher, R. Inkulu, K. Pingali, and C. Casçaval, “How much parallelism is there in irregular applications?” in *ACM sigplan notices*, vol. 44, no. 4. ACM, 2009, pp. 3–14.
- [7] Y. Etsion, F. Cabarcas, A. Rico, A. Ramirez, R. M. Badia, E. Ayguade, J. Labarta, and M. Valero, “Task superscalar: An out-of-order task pipeline,” in *Proceedings of the 2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE Computer Society, 2010, pp. 89–100.
- [8] R. A. Van Engelen, “Efficient symbolic analysis for optimizing compilers,” in *International Conference on Compiler Construction*. Springer, 2001, pp. 118–132.
- [9] K. Kennedy and J. R. Allen, *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc., 2001.
- [10] D. C. S. Lucas and G. Araujo, “The batched doacross loop parallelization algorithm,” in *High Performance Computing & Simulation (HPCS), 2015 International Conference on*. IEEE, 2015, pp. 476–483.
- [11] R. Cytron, “Doacross: Beyond vectorization for multiprocessors,” in *Proc. of the Int. Conf. on Parallel Processing, 1986*, 1986.
- [12] P. Unnikrishnan, J. Shirako, K. Barton, S. Chatterjee, R. Silvera, and V. Sarkar, “A practical approach to doacross parallelization,” in *European Conference on Parallel Processing*. Springer, 2012, pp. 219–231.
- [13] J. Jaeger, P. Carribault, and M. Pérache, “Fine-grain data management directory for openmp 4.0 and openacc,” *Concurrency and Computation: Practice and Experience*, vol. 27, no. 6, pp. 1528–1539, 2015.
- [14] J. Ferrante, K. J. Ottenstein, and J. D. Warren, “The program dependence graph and its use in optimization,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 9, no. 3, pp. 319–349, 1987.

- [15] L. Rideau, B. P. Serpette, and X. Leroy, “Tilting at windmills with coq: formal verification of a compilation algorithm for parallel moves,” *Journal of Automated Reasoning*, vol. 40, no. 4, pp. 307–326, 2008.
- [16] O. Bachmann, P. S. Wang, and E. V. Zima, “Chains of recurrences—a method to expedite the evaluation of closed-form functions,” in *Proceedings of the international symposium on Symbolic and algebraic computation*. ACM, 1994, pp. 242–249.